翁恺课 · 第 6 主题

W6 数组与函数

对齐翁恺《程序设计入门——C语言》第 6 讲 · 目标：会用函数封装可复用代码（声明/定义/调用）、分得清「传值 vs 数组传址」、会把数组操作封装成函数 · 不覆盖：数组运算进阶、指针（W7/W8）

6.1 为什么要函数：代码也会"复制粘贴"

前面你把求最大、求和写了一遍又一遍——每次换个数据就重抄一遍。程序一大，重抄就是噩梦：改一处错处要改好几处、代码又长又乱。翁恺的说法：**函数 = 给你的代码块"起名字 + 打包"**，写一次，处处能用。就像洗衣机的"程序键"：按一下"快洗"，机器自己跑那一套动作，你只管按键。

> **一句话：**函数把一段能重复用的代码**打包**，给它**名字**、给它**参数**（要喂什么）和**返回值**（吐出什么）。C 程序就是 **main 加上一堆被它调用的函数**。

6.2 函数"三件套"：声明 / 定义 / 调用

写一个函数要凑齐三样东西，缺一样编译就报错：

名字	作用	长什么样
声明（prototype）	提前告诉编译器"有这么一个函数"（名字 + 参数类型 + 返回类型）	`int max_of(int a, int b);`
定义（definition）	写函数体，说明"它到底怎么算"	`int max_of(int a, int b){ ... }`
调用（call）	在别处（通常是 main）把它用起来	`max_of(x, y)`

> **新手坑：忘声明 = 隐式声明报错。**C99 起不允许"先用再声明"。如果没写声明就直接调用，gcc 会报 `error: implicit declaration of function`。最稳的写法：**声明放前面（或函数定义放 main 之前）**，main 后面再写定义。

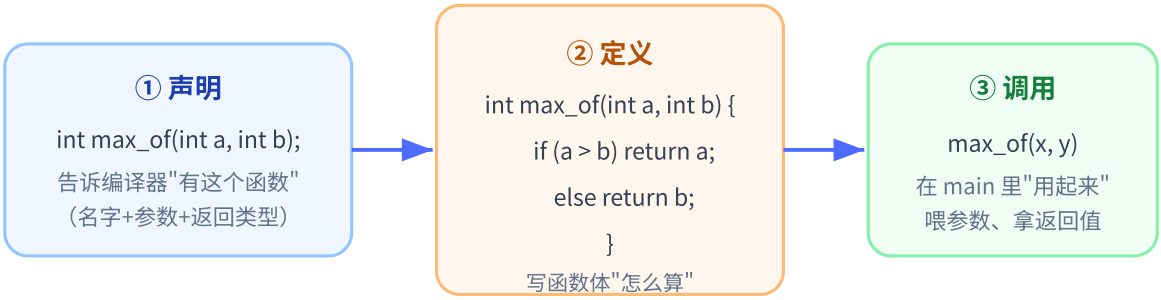


图 W6-1　函数三段式：声明 → 定义 → 调用。声明告诉编译器形状，定义写算法，调用真正用起来

6.3 函数封装：求两数的较大值

最典型的封装：把"求较大值"打包成 `max_of`，main 只管喂两个数、拿结果：

```
/* wk6_max.c — 函数封装：求两个数的较大值（gcc 实跑）*/
#include <stdio.h>

int max_of(int a, int b);   /* 声明 */

int main(void)
{
    int x = 7, y = 3;
    printf("较大的是 %d\n", max_of(x, y));
    return 0;
}

int max_of(int a, int b)    /* 定义 */
{
    if (a > b) return a;
    else       return b;
}
```

实跑输出：

```
较大的是 7
```

> **翁恺提醒：**调用时 `x`、`y` 是**实参**（main 里的真值），函数里 `a`、`b` 是**形参**（占位）。函数拿到的是实参的**一份拷贝**，改形参改不到实参——这正是下一节"传值"的核心。

6.4 传值：普通变量作参数，改的是副本

C 里普通变量作参数是**传值（pass by value）**：函数拿到的是实参的**拷贝**，在函数里怎么改都**不影响**外面的变量。最经典的例子是写一个 swap 交换两个数——你会发现**根本换不动**：

```
/* wk6_passval.c — 传值演示：swap 改不到外面的 x,y（经典坑）*/
#include <stdio.h>

void swap(int a, int b);   /* void = 不返回东西 */

int main(void)
{
    int x = 3, y = 5;
    printf("调用前：x=%d y=%d\n", x, y);
    swap(x, y);
    printf("调用后：x=%d y=%d（没变！）\n", x, y);
    return 0;
}

void swap(int a, int b)    /* 传值：改的是副本 */
{
    int t = a;
    a = b;
    b = t;
}
```

实跑输出：

```
调用前：x=3 y=5
调用后：x=3 y=5（没变！）
```

> **新手坑：想用函数改外面的变量，普通传值做不到。**swap 里确实把 a、b 换了，但那只是 x、y 的**拷贝**，换完 x、y 原封不动。想真正交换怎么办？两条路：①把交换逻辑写在 main 里（不封装）；②学 W8 用**指针**传地址。记住：**传值 = 复制一份，改不动原件**。

6.5 数组作参数：传的是首地址（传址语义）

数组作参数和普通变量**不一样**：函数收到的**不是整份拷贝**，而是数组的**首地址**（即"第一个元素在哪"），函数通过地址访问的是**原数组本身**。所以——**在函数里改数组，会改到 main 里的原数组**：

```
/* wk6_arrparam.c — 数组作参数：函数内改会改原数组（gcc 实跑）*/
#include <stdio.h>

void print_array(int a[], int n);
```

```
void bump_array(int a[], int n);   /* 每个元素 +1 */

int main(void)
{
    int score[5] = {60, 70, 80, 90, 100};
    print_array(score, 5);
    bump_array(score, 5);
    printf("调用 bump 之后：");
    print_array(score, 5);
    return 0;
}

void print_array(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bump_array(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++) a[i] += 1;  /* 改的是原数组！*/
}
```

实跑输出：

```
60 70 80 90 100
调用 bump 之后：61 71 81 91 101
```

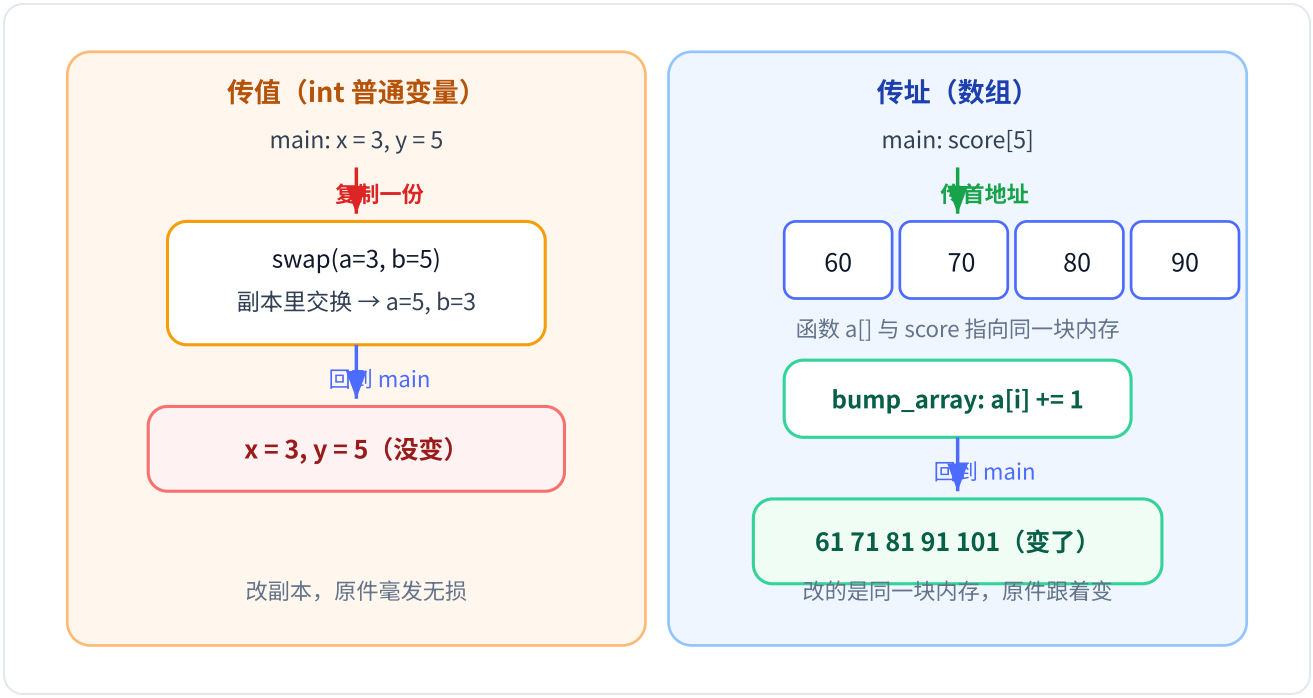


图 W6-2　传值 vs 传址：int 参数复制一份、函数内改不动原件（swap 换不掉）；数组参数传首地址、函数内改的就是原数组

6.6 封装成函数：sum_array / max_array

把 W5 学到的"数组求和、求最大"打包成函数，main 就清爽了：

```
/* wk6_arrfun.c — 封装成函数：数组求和 + 求最大（gcc 实跑）*/
#include <stdio.h>

int sum_array(int a[], int n);
int max_array(int a[], int n);

int main(void)
{
    int a[5] = {4, 9, 2, 7, 5};
    printf("总和 = %d，最大值 = %d\n", sum_array(a, 5), max_array(a, 5));
    return 0;
}

int sum_array(int a[], int n)
{
    int i, s = 0;
    for (i = 0; i < n; i++) s += a[i];
    return s;
}

int max_array(int a[], int n)
{
    int i, m = a[0];
```

```
    for (i = 1; i < n; i++) {
        if (a[i] > m) m = a[i];
    }
    return m;
}
```

实跑输出：

```
总和 = 27，最大值 = 9
```

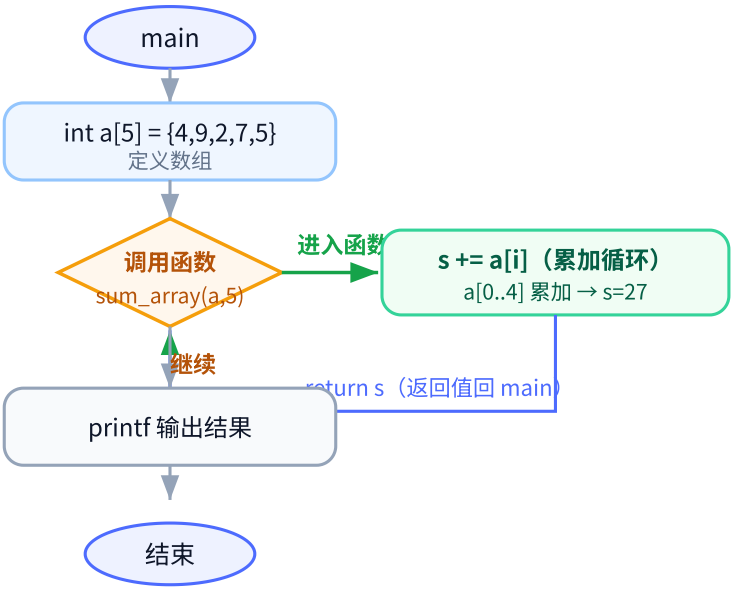


图 W6-3　main 调用函数流程：main 喂数组和长度 → 进入函数跑累加循环 → return 结果回 main → 输出。函数像个"黑盒"，只管喂进去、拿回来

6.7 新手坑集中营（本讲最容易错的 3 件事）

1. **非 void 函数漏 return** → 返回类型写了 `int` 却忘 `return 值;`，函数"走到头没吐出东西"，返回值是垃圾。gcc 实测警告：

```
wk6_bug_ret.c:16:1: warning: control reaches end of non-void function [-Wreturn-type]
   16 | }
      | ^
```

2. **忘声明就调用（隐式声明）** → C99 起直接 `error: implicit declaration of function`。修：把声明写前面，或函数定义放 main 之前。
3. **以为传值能改外面** → 普通变量作参数是复制一份，`swap(x, y)` 换不动。想改外面的量，本讲先记住"做不到"，W8 指针才有解。

6.8 一分钟自测（答案见练习册）

判断/填空：

1. 写一个函数要凑齐 ____、____、____ 三样东西；返回值类型是 `void` 表示 ____。
2. 普通变量作参数是 ____（传值/传址），函数内改形参 ____ 影响实参；数组作参数传的是 ____，函数内改数组 ____ 影响原数组。
3. 非 void 函数漏写 `return`，gcc 给 ____ 警告；这是____（语法错误/警告）。

6.9 练习指引

去配套练习册做 W6 练习。翁恺建议本讲里程碑：**把一个"求数组和/最大/平均"的完整程序重写成函数版**——main 只管定义数据 + 调用 + 打印，所有计算都搬进函数。这样你的 main 会短得"像一份使用说明书"，这就是函数的意义。

本讲义为自写原创讲解（对齐翁恺《程序设计入门——C语言》第 6 讲），仅作课堂伴读；示例代码均用 gcc -Wall -Wextra 实测通过（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。视频与图片版权归原作者所有，本讲义不含课程原素材。